

Anomaly Detection on Unstable Logs with GPT Models

Fatemeh Hadadi¹, Qinghua Xu², Domenico Bianculli³, and Lionel Briand^{1,2}

¹University of Ottawa, Ottawa, Canada

{fhada072@uottawa.ca, lbriand@uottawa.ca}

²Lero centre and University of Limerick, Limerick, Ireland et .qinghua@gmail.com

³University of Luxembourg, Luxembourg, Luxembourg domenico.bianculli@uni.lu

June 12, 2024

Abstract

Log-based anomaly detection has been widely studied in the literature as a way to increase the dependability of software-intensive systems. In reality, logs can be unstable due to changes made to the software during its evolution. This, in turn, degrades the performance of downstream log analysis activities, such as anomaly detection. The critical challenge in detecting anomalies on these unstable logs is the lack of information about the new logs, due to insufficient log data from new software versions.

The application of Large Language Models (LLMs) to many software engineering tasks has revolutionized various domains. In this paper, we report on an experimental comparison of a fine-tuned LLM and alternative models for anomaly detection on unstable logs. The main motivation is that the pre-training of LLMs on vast datasets may enable a robust understanding of diverse patterns and contextual information, which can be leveraged to mitigate the data insufficiency issue in the context of software evolution. Our experimental results on the two-version dataset of LOGEVOL-Hadoop show that the

fine-tuned LLM (GPT-3) fares slightly better than supervised baselines when evaluated on unstable logs. The difference between GPT-3 and other supervised approaches tends to become more significant as the degree of changes in log sequences increases. However, it is unclear whether the difference is practically significant in all cases. Lastly, our comparison of prompt engineering (with GPT-4) and fine-tuning reveals that the latter provides significantly superior performance on both stable and unstable logs, offering valuable insights into the effective utilization of LLMs in this domain.

Keywords: unstable logs, anomaly detection, large language model, GPT

1 Introduction

Over the past decade, various software-intensive systems, such as online service systems and big data systems, have permeated every aspect of people's daily lives. As the prevalence of such systems continues to grow, the potential impact of software failures has become increasingly significant. A critical software failure can result in service interruptions,

financial losses, and, in severe cases, pose threats to human safety (32).

Log-based anomaly detection has emerged as a widely explored approach to enhancing the dependability of software-intensive systems. An anomaly detector aims to discern irregular patterns within system logs, which serve as vital indicators of the system’s operational state. Early research predominantly employed classical machine learning techniques, such as Principal Component Analysis (73), Isolation Forest (43), and one-class SVM (21) for automatic anomaly detection. These methods are primarily characterized by their reliance on manual feature engineering, wherein features are extracted from log data based on domain knowledge and subsequently used as input for a supervised machine learning classifier, such as decision trees, random forests, or SVM. In addition to supervised approaches, researchers have also proposed semi-supervised and unsupervised methods, which leverage the inherent structures and patterns within the logs with limited labeled data (74; 46; 42; 12; 52; 75).

As the volume and complexity of logs continue to grow, manual feature engineering has become difficult, time-consuming, or even infeasible in some cases (53). In response, deep learning methods have garnered increasing attention for their capability to automate feature extraction. Specifically, log-based anomaly detection has significantly benefited from sequential deep learning models such as LSTM and transformers, as logs are inherently presented as semi-structured or unstructured text sequences (52; 69; 14; 12).

However, most existing log-based anomaly detectors operate under a closed-world assumption, where logs remain stable in terms of structure and content. In reality, logs can be *unstable* due to changes made to the software during its evolution. Software systems evolve, generating logs that reflect these changes. These logs may thus undergo alterations

such as addition, removal, or modification of individual log messages as well as changes to the order of these messages. Such changes render these logs unstable, thus potentially undermining the performance of methods trained only with stable logs. Specifically, the task of anomaly detection on unstable logs (hereafter ADUL) involves identifying anomalies in logs that continuously evolve due to software evolution. One straightforward solution is to train a new anomaly detector from scratch with unstable logs. However, the required data is not always available, e.g., in the case of a newly deployed software producing insufficient data for deep learning training.

We contend that a promising approach to addressing data insufficiency in ADUL is the application of Large Language Models (LLMs). The emergence of LLMs has revolutionized various domains, creating hype around applying them to nearly every task involving textual data (78). The main motivation for applying LLMs in ADUL is that the pre-training on vast datasets may enable a robust understanding of diverse patterns and contextual information, which can be leveraged to mitigate the data insufficiency issue in the context of ADUL.

Two prevalent strategies to utilize LLMs, which are promising for ADUL, are *prompt engineering* and *fine-tuning*. Liu et al. (44) proposed LogPrompt, a *prompt engineering*-based method for log analysis. LogPrompt first crafts an effective prompt. A typical prompt should include task description, expected input/output, and examples relevant to the task (e.g., samples from stable logs for ADUL). This prompt is then given as input to a pre-trained LLM, such as GPT-3 (2), for anomaly detection. *Fine-tuning*, on the other hand, first gathers a relevant training dataset to fine-tune the LLM with log sequences and their corresponding labels, rather than solely relying on the LLM’s internal knowledge. Afterwards, an instructive prompt is presented to the newly trained LLM for log-based anomaly de-

tection. To our knowledge, fine-tuning LLMs for ADUL has still not been explored by any published research. Consequently, a comprehensive comparison of prompt engineering, fine-tuning, and alternative deep learning models for ADUL is warranted.

To fill in this gap, we first propose a general framework to fine-tune an LLM for ADUL. Then, we implement this framework by fine-tuning GPT-3 and report on an experimental comparison of fine-tuning, prompt engineering, and nine traditional deep learning methods, using two public datasets and a synthetic one. Experimental results lead to the following key findings (1) Fine-tuned GPT-3 fares slightly better than supervised baselines for ADUL on the two-version dataset of LOGEVOL-Hadoop. (2) As the degree of changes in logs increases, the difference between the fine-tuned GPT-3 and other supervised approaches tends to become more significant. (3) The comparison between fine-tuned GPT-3 and prompt-engineered GPT-4 shows that the former provides significantly superior performance on both stable and unstable logs, offering valuable insights into the effective utilization of LLMs on ADUL. The choices of GPT3 and GPT-4 for the tasks above will be explained.

2 Background

2.1 Logs and Unstable Logs

Logs are semi-structured or unstructured text generated by logging statements (e.g., `logging.info('Received Block %d from %s', id, ip)`) in source code (20). The main concepts related to logs are *log message*, *log message sequence*, *log template* and *log template sequence*, which we explain below and exemplify in Figure 1.

A *log message* contains two main components: the *header* (e.g., timestamp or log level), and the *con-*

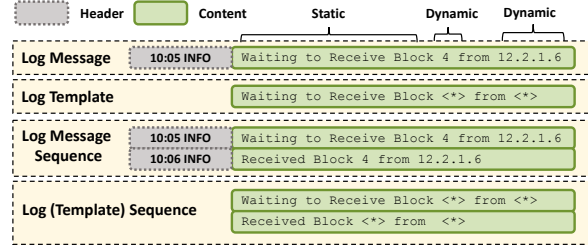


Figure 1: Examples of log Message, Log Message Sequence, Log Template and Log Template Sequence.

tent, depicted as grey-dotted and green boxes in Figure 1, respectively. The content of a log message can be further divided into static and dynamic parts. The static parts refer to the fixed text written by developers in the logging statement, e.g., “Waiting to Receive Block from” and “Received Block from”; the dynamic parts are expressions evaluated at runtime, such as the actual block id 4 and IP address 12.2.1.6.

A *log template*, also called *event template* or *log key*, is usually obtained through log parsing (76), which masks the dynamic parts of the log message content with a special symbol, such as `<*>`. Compared to log messages, log templates eliminate the influence of specific values in the dynamic parts, enabling downstream tasks (e.g., anomaly detection (33), log summarization (45)) to focus on analyzing patterns within logs, without being distracted by concrete values.

A *log message sequence* is a log fragment consisting of multiple log messages, which typically records the execution flow of a specific job or process. A log message sequence consists of log messages that either pertain to a specific task (i.e., session-based partitioning) or are grouped within a fixed-size window (i.e., sliding/fixed window partitioning). Session-based partitioning groups log messages based on their session IDs, thereby creating log

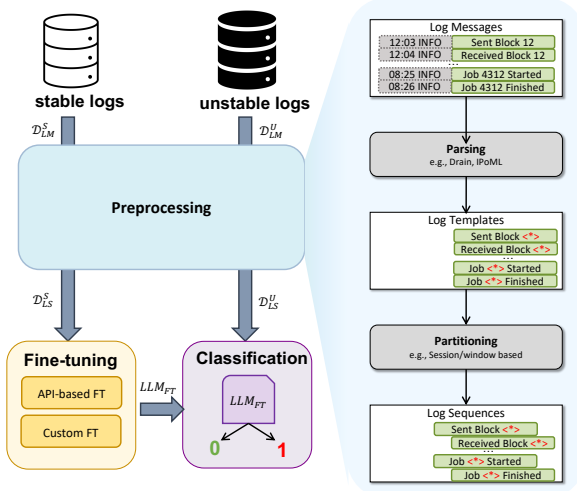


Figure 2: Overview of fine-tuning an LLM for ADUL

sequences that encapsulate the activities within each session. On the other hand, sliding/fixed window partitioning employs a fixed-size window to group log messages, generating log sequences that capture a snapshot of system activity over time.

Parsing a log message sequence yields a *log template sequence* (as shown at the bottom of Figure 1). We remark that “log template sequence” is often referred to simply as “log sequence” in the literature; therefore, for brevity, we adopt the term *log sequence* throughout this paper.

Unstable logs are logs produced due to the changes made during software evolution. Concretely, developers often modify source code including logging statements, which in turn result in the changes of logs. As Kabinna et al. (30) reported, around 25%-40% log statements changed during their lifetime. The changes of log statements produce logs with new log templates and log sequences.

During system updates, evolution in software can manifest *unstable logs* at two main levels: (1) log sequence level and (2) log template level (77). On the

log sequence level, changes in software functions or the positioning of logging statements can disrupt log sequences. This may result in the addition (or removal) of log templates or in changes in the order of log sequences. On the template level, modifications to the logging statements—such as additions, removals, or modifications—alongside errors in log parsing, can create instabilities in the log templates themselves.

2.2 Anomaly Detection on Logs

Anomalies in logs refer to logs that do not conform to the normal behavior of a system (20). Log-based anomaly detection, which is typically referred to as *anomaly detection on stable logs* (ADSL hereafter), is a binary classification task (20). Let $\mathcal{D}_{LS}^S$ be the stable log sequence dataset. The goal of ADSL is to learn a binary classification function that predicts the labels $ls_i^S \in \{0, 1\}$ for each log sequence $ls_i^S \in \mathcal{D}_{LS}^S$, where 0 and 1 represent normal and anomalous sequences, respectively.

ADUL considers log changes when a software system evolves from one version to another. In the case of such unstable logs, insufficient labeled logs from the evolved software may pose a challenge for retraining or rebuilding an anomaly detection model. Thus, alternatively, ADUL aims to learn a binary classification function f from the stable log sequence dataset $\mathcal{D}_{LS}^S$ and predict anomalies in unstable log sequences $\mathcal{D}_{LS}^U$ with minimal errors.

It is worth noting that in this paper, we use log sequences as the input for anomaly detection instead of raw log message sequences. The reason for this practice is two-fold. First, parsing raw log message sequences into log sequences significantly reduces their length since variable-length dynamic parts are replaced by a short symbol, i.e., “<*>”. The reduced length can mitigate the input token limit for LLMs. Second, according to Huang et al. (24),

anomaly detection using log sequences can enhance effectiveness on logs that are stable or have low instability (under a 20% injection ratio). However, the dynamic parts of raw log message sequences, such as IP addresses, can cause deep learning models to overfit. This issue is particularly pronounced in logs with higher instability because the variability in these dynamic parts increases, leading to a greater risk of overfitting and reduced generalization.

3 Anomaly Detection on Unstable Logs via Fine-tuning LLMs

The ADUL task involves learning a binary classification function from a dataset of stable logs and predicting the labels for unstable logs. This section describes our proposed framework, which performs ADUL *by fine-tuning an LLM*. Figure 2 depicts the overview of our framework, which consists of three components, namely preprocessing, fine-tuning, and classification. The *preprocessing component* transforms the stable log messages in $\mathcal{D}_{LM}^S$ into log sequences $\mathcal{D}_{LS}^S$, which are fed to the *fine-tuning component* to produce a fine-tuned LLM, denoted by LLM_{FT} . Similarly, the *preprocessing component* transforms the unstable log message dataset $\mathcal{D}_{LM}^U$ into the log sequence dataset $\mathcal{D}_{LS}^U$. In the *classification component*, LLM_{FT} predicts the label $l_i^U \in \{0, 1\}$ of each unstable log sequence $ls_i^U \in \mathcal{D}_{LS}^U$, where 0 indicates a normal sequence and 1 an anomalous one. Further details of each component are provided below.

The *preprocessing component* is illustrated in detail at the bottom of Figure 2, comprising two sub-components, namely parsing and partitioning. Given a raw log message (e.g., “12:03 INFO Sent Block 12”), we leverage a parser (e.g., Drain (17)), to identify the static parts (e.g., “Sent Block”) and dynamic parts (e.g., “12”) and re-

place the latter with the symbol $\langle * \rangle$. The parsing sub-component yields log templates, which are then fed into the partitioning sub-component. This component aggregates log templates into log sequences based on their session IDs or a fixed-size window (as described in § 2.1). In general, session-based partitioning is favoured, as long as the maximum sequence length does not exceed the input limit, since session-based partitioning is reported to yield better results than the window-based partitioning on the same dataset for anomaly detection (36).

The *fine-tuning component* aims to adapt a pre-trained, general-purposed LLM to a specific task, i.e., ADUL in our context. As shown in Figure 2, the *fine-tuning component* includes two types of fine-tuning (FT) techniques, namely API-based FT and Custom FT. API-based FT refers to fine-tuning performed through dedicated APIs made available by the LLM provider, e.g., OpenAI (56). This is typically the case for closed-source LLMs, such as GPT-3 (2), GPT-3.5 (58), and GPT-4 (47), for which neither full nor selective fine-tuning is allowed without accessing their APIs. These APIs support fine-tuning on a set of prompt-completion pairs or conversations depending on whether LLMs are used in purely generative or conversational settings. Custom FT, on the other hand, is applicable to open source LLMs, such as LLama (65) and Bert (11). Common custom FT techniques include full fine-tuning and parameter-efficient fine-tuning (PEFT). There are several types of PEFT methods, including additive, selective, reparameterized, and hybrid PEFT (16). The predominant PEFT techniques are LoRa (23) and its derivative techniques, which construct a low-dimensional parameterization of LLMs’ parameters for fine-tuning. Such techniques significantly reduce the computational cost while maintaining comparable performance as fully fine-tuned LLMs.

Finally, the *classification component* leverages the

LLM_{FT} model produced by the *fine-tuning component* to perform ADUL. Concretely, we rely on the *preprocessing component* to transform the stable log message dataset $\mathcal{D}_{LM}^U$ into the unstable log sequence dataset $\mathcal{D}_{LS}^U$. LLM_{FT} takes an unstable log sequence $ls_i^U \in \mathcal{D}_{LS}^U$ as input and attempts to accurately predict its label $l_i^U \in \{0, 1\}$.

4 Evaluation

We aim to make a realistic appraisal of LLMs to support ADUL. We thus select one of the most representative LLMs for our study. We considered the series of GPT models (2; 1), which are the major driving force behind widely used AI applications (e.g., ChatGPT).

Although, at the time of writing, the most recent GPT model available for fine-tuning is GPT-3.5 (58), given space constraints and the fact that we obtained better results with GPT-3 than GPT-3.5, we only report results for the former. For the same reason, for our experiments involving prompt engineering, when no fine-tuning is performed, we rely on the latest model, GPT-4 (47), which yields the best results compared to earlier GPT models¹.

More specifically, we report on a comprehensive evaluation of the performance of various methods on the ADUL task, including fine-tuned GPT-3 (denoted by GPT-3_{FT} hereafter), prompt engineered GPT-4 (denoted by GPT-4_{PE} hereafter) and nine traditional machine learning as well as deep learning methods (hereafter collectively called *ML+DL methods*) that we use as baselines. We first assess the performance of GPT-3 for ADUL and ADSL by comparing GPT-3_{FT} with ML+DL methods. Although ADSL is not the focus of our study, we add it as a

base of comparison whenever both stable and unstable testing sets are available. Then, we investigate which, of fine-tuning and prompt engineering, is a superior strategy. More precisely, in RQ1, we assess the overall effectiveness of GPT-3_{FT} and baselines. In RQ2, we further investigate the effectiveness and robustness of GPT-3_{FT} and baselines under different evolution scenarios, such as different changes at the log-sequence level and different percentages of log changes. In RQ3, we compare GPT-3_{FT} with two prominent types of prompt engineering, namely zero-shot and few-shot prompt engineering, based on GPT-4.

Summing up, we investigate the following research questions:

- RQ1** How does the overall effectiveness of GPT-3_{FT} compare to ML+DL methods for the ADSL and ADUL tasks?
- RQ2** How effective and robust are GPT-3_{FT} and ML+DL methods under different log evolution scenarios, including varying changes in log statements and different percentages of log changes?
- RQ3** Which strategy is superior when using GPT for the ADUL task: fine-tuning or prompt engineering?

4.1 Experiment Setup and Implementation

4.1.1 Datasets

We used two public datasets and a synthetic one, namely LOGEVOL-Hadoop (25)², HDFS (72) and SynHDFS. We excluded other relevant datasets from the literature (25), such as LOGEVOL-Spark and SYNEVOL, because their log sequences exceeded the input size limit of GPT-3; such a limit could

¹Additional results of fine-tuning with GPT-3.5 and prompt engineering with GPT-3.5 are reported in the replication package.

²Since the replication package URL (26) was empty when this work started, we obtained the dataset by directly contacting the authors.

Table 1: Overview of Datasets

Name	Sys	Synthetic	#Log Messages	#Anomalous Messages	#Sessions	#Log Templates	Session Length		
							avg	min	max
LOGEVOL	Hadoop 2	No	2,120,739	35,072 (1.6%)	333,699	319	6.35	1	1963
	Hadoop 3	No	2,050,488	30,309 (1.4%)	343,013	313	5.97	1	1818
HDFS	Hadoop	No	11,110,850	284,818 (2.9%)	575,061	48	19.32	2	30
SynHDFS	Hadoop	Yes	1,020,922	-	51,000	38	20.01	1	52

not even be circumvented by breaking the log sequence into smaller partitions (and then manually relabelling each smaller partition) since labels are unavailable at the log message level. We also excluded other available datasets for anomaly detection, such as BGL (55) and Sprite (55), because they are not intended for unstable log studies derived from software evolution.

Table 1 presents relevant datasets statistics; column “Sys” indicates the system from which the logs were collected. Column “Synthetic” indicates whether the log data is original or synthesized. “#Log Messages”, “#Anomalous Messages”, “#Sessions”, and “#Log Templates indicate the number of log messages, anomalous log messages, sessions, and unique log templates in each dataset, respectively. Column “Session Length” indicates the average, minimum, and maximum number of log messages in each session.

LOGEVOL-Hadoop Introduced recently by Huo et al. (25), LOGEVOL-Hadoop is a dataset that captures real-world system logs from two versions of Hadoop (63) (i.e., versions 2.10.2 and 3.3.3, referred to as Hadoop 2 and 3, respectively). Evolving from Hadoop 2 to Hadoop 3 induces log evolution in both template and sequence levels (defined in § 2.1). Logs are generated using HiBench (27), a big data benchmark suite, during the operation of twelve tasks, including sorting and classification (19; 42). To capture real-world anomaly scenarios

into logs, they injected 18 fault types into the system, including network fault, process suspension, process killing, and resource occupation. As shown in Table 1, Hadoop 2 contains 2,120,739 log messages, of which 1.3 % are anomalous. Similarly, Hadoop 3 comprises 2,050,488 log messages, with a slightly higher anomaly rate of 1.4 %. We further describe the distinction between these two versions in terms of log template evolution. Specifically, 104 (33.22 %) of Hadoop 3’s unique log templates have not been observed in Hadoop 2. Similarly, 110 (34.48 %) of Hadoop 2’s unique log templates are absent in Hadoop 3.

HDFS Hadoop Distributed File System (HDFS) logs (72) are produced by running Hadoop-based MapReduce jobs on Amazon EC2 nodes, consisting of 11,197,954 log messages, of which 284,818 (2.9 %) are manually labeled anomalous. The average number of log messages in a sequence is 19.32. The total number of unique log templates is 48.

SynHDFS This dataset contains unstable log sequences, synthesized by applying changes to the HDFS dataset. Notably, we only apply sequence-level changes in SynHDFS, excluding template-level changes due to a lack of implementation details from previous studies (77; 24; 40). Specifically, according to the empirical study of Zhang et al. (77), three types of template level changes can be adapted to reflect system evolution: (1) removing a log template,

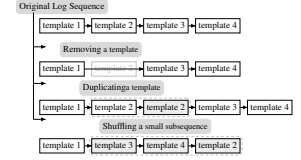


Figure 3: Synthetic Log Sequence Examples

(2) duplicating a log template, or (3) shuffling a subsequence, as illustrated in Figure 3. However, such changes can potentially transform a normal log sequence into an anomaly or vice versa, rendering its label inaccurate. As a remedy, we selectively apply changes only to log templates that are less likely to flip their labels because of such changes. We adopt the selection strategy proposed by Xu et al. (74), which combines a decision tree and manual examination for selection. To reduce the cost of manual examination, we sampled and applied changes to a subset of the HDFS dataset instead of the full dataset.

Concretely, we randomly selected 50,000 normal and 1,000 anomalous log sequences, following the settings in the study of Zhang et al. (77), to keep the anomaly percentage (2%) close to that of the original HDFS dataset. We applied injection ratios of 5%, 10%, 20%, and 30% to produce four datasets, following common practice in the literature (77; 24; 40; 25). The last row of Table 1 presents the statistics for SynHDFS. It includes 1,020,922 log messages. Since labels in the original HDFS dataset are only available at the session level, the number of anomalous log messages is unavailable. The average, minimum, and maximum length of log sequences are 20.1, 1, and 52, respectively.

4.1.2 Baselines

We considered nine ML+DL methods as baselines in this paper, including five unsupervised, one semi-supervised, and three supervised. Among these methods, NeuralLog (34) achieves state-of-the-art performance on the ADSL task. However, the leading approach for ADUL remains undetermined as different evaluation datasets are used in their respective studies. Our choice of baselines is also determined by source code availability to ensure the implementation’s reliability. Consequently, we had to exclude models such as SwissLog (40), Hi-

tAnomaly (24), and EvLog³ (3). We adopted the implementation of baseline models from code provided by Le and Zhang (36) and He et al. (18).

Table 2 shows the main characteristics of the ML+DL methods; we provide a brief description in the following. *Principal Component Analysis (PCA)* (74) converts logs into count vectors (10) and then uses the PCA algorithm to detect the label of log sequences by assigning them to either the normal or anomalous space. *Invariant Mining (IM)* (46) mines invariants from log count vectors and classifies them as anomalous or normal log sequences. *LogCluster* (42) clusters log sequences by computing the similarity of log representations to the centroid of normal logs. *DeepLog* (12) applies two layers of long short-term memories (LSTMs) in their network (22) to predict the next event from a given log sequence and labels sequences as anomalous if the predicted log is different than the actual log template. *LogAnomaly* (52) has an architecture similar to DeepLog, further improved by adopting semantic embeddings for log templates and adding an attention layer between LSTM layers. *PLELog* (75) is a semi-supervised strategy that uses normal data as well as a small subset of unlabeled data to train. First, it adopts a clustering method (HDBSCAN (51)) to probabilistically predict the labels of unlabeled data and then uses them to train an attention-based GRU (6) to detect anomalies. *LogRobust* (77) uses a pre-trained word vectorizer (FastText (29)) to extract semantic information from log templates and utilizes an attention-based BiLSTM model (60) to detect anomalous log sequences. *CNN* (48) transforms an input log sequence into a trainable matrix and uses this matrix as input to train a Convolutional Neural Network (31; 38) for log-based anomaly detection. *NeuralLog* (34) employs

³As remarked above, the EvLog’s replication package URL (26) was empty.

Table 2: Overview of Baselines

Learning Method	Approach	Parser	Log Representation	ML Method	Base Model
Unsupervised	PCA	Yes	Template ID	Traditional ML	PCA
	IM	Yes	Template ID	Traditional ML	IM
	LogCluster	Yes	Template ID	Traditional ML	Clustering
	DeepLog	Yes	Template ID	Deep Learning	LSTM
	LogAnomaly	Yes	Template2Vec	Deep Learning	LSTM
Semi-supervised	PLELog	Yes	FastText and TF-IDF	Deep Learning	GRU
Supervised	LogRobust	Yes	FastText and TF-IDF	Deep Learning	BiLSTM
	CNN	Yes	Logkey2vec	Deep Learning	CNN
	NeuralLog	No	BERT	Deep Learning	Transformer

BERT (11) to enhance the sequence representations used to detect anomalies through a transformer-based classification model (67).

p-value is below the predefined significance level, usually set at 0.05, the difference is considered statistically significant.

4.1.3 Evaluation Metrics and Statistical Testing

To measure the effectiveness of GPT-based anomaly detection models, we rely on Precision, False Positive Rate, Recall, and F1-score as evaluation metrics. We consider TP (true positive) as the number of anomalies that are correctly detected by the model, FP (false positive) as the number of normal log sequences that are labeled as anomalous by the model, and FN (false negative) as the number of anomalous log sequences that the model does not detect. Precision (P) is calculated by $\frac{TP}{TP+FP}$ as the percentage of true anomalies among all anomalies detected by the model. Recall (R) is the percentage of anomalies in the data being detected, computed by $\frac{TP}{TP+FN}$. F1 score (F1) is the harmonic mean of Precision and Recall, i.e., $2 * \frac{P * R}{P + R}$. To assess the statistical significance of our results, we use Fisher’s Exact Test (66), which determines the likelihood of the observed data if there are no real differences in anomaly detection rates between models. Fisher’s Exact Test compares two methods A and B and calculates a p-value. If this

4.1.4 Implementation Details

The number of epochs and batch size for fine-tuning GPT-3 are set to 3 and 15, respectively, which are automatically configured by the fine-tuning algorithm, as recommended by OpenAI. We use the babbage-002 version of GPT-3 and the GPT-4-turbo version of GPT-4, following OpenAI’s recommendation due to their low cost (56; 57). The maximum log sequence length of LOGEVOL-Hadoop is 1,963, which exceeds the input limit of GPT-3. To circumvent this issue, we adopt the sliding window partitioning strategy, as described in § 2.1, grouping log messages into small partitions that comply with the input limit of GPT-3. We performed session-based partitioning for HDFS with the block_id in each log message. However, since only a small fraction of sessions (approximately 3.5 %) contains more than 30 log templates, we chose to follow the implementation by Le and Zhang (36) and truncate these long sessions to ensure that all sessions are within the 30-template limit. This approach

allows us to maintain manageable input sizes for GPT models while still capturing most of the session information. For baselines, hyper-parameters are set as reported in their original papers or suggested by their implementation packages. LogAnomaly’s representation model (template2vec) requires domain-specific antonyms and synonyms for training. This information is unavailable and, thus, similar to (36), we used a pre-trained FastText model (29) to compute the semantic vectors. As our training sets are too large to process on NeuralLog, we used a subset containing the first 200,000 log messages, following the methodology adopted in prior work (36).

A prefix graph parser (7) is adopted for the LOGEVOL-Hadoop dataset, following its original paper, while for HDFS and SynHDFS we adopt the state-of-the-art, publicly-available log parser Drain (17). LOGEVOL-Hadoop and HDFS are split into training and testing with an 80:20 split ratio. SynHDFS, as a synthesized dataset of the HDFS, is only used as testing set. We are aware that chronological splitting is preferred in the literature for reducing the risk of data leakage (36). However, in this paper, we split training and testing datasets randomly due to the absence of timestamp information. In creating fine-tuning datasets, due to the high cost of using all training sets, we randomly undersample the majority class (normal log sequences) to reduce their size.

To reduce the influence of randomness in results, we ran each experiment five times and calculated the mean value of the results for each experiment. Furthermore, we set the temperature, a parameter controlling the randomness of GPT responses, to its lowest value, 0. We conducted all experiments with cloud computing environments provided by the Digital Research Alliance of Canada (54) on the Cedar cluster with 94,528 CPU cores for computation and 1,352 GPU devices. We plan to make the replication package, including our synthesized dataset

Table 3: Effectiveness of GPT-3_{FT} and ML+DL methods for ADUL and ADSL with the LOGEVOL-Hadoop dataset (training set → testing set)

Learning Method	Model	2 → 2			2 → 3		
		P	R	F1	P	R	F1
supervised	GPT-3 _{FT}	0.997	0.999	0.998	0.980	0.999	0.989
	NeuralLog	0.996	0.998	0.997	0.996	0.925	0.959
	LogRobust	0.995	0.998	0.997	0.963	0.999	0.981
	CNN	0.999	0.998	0.999	0.925	0.995	0.959
semi-supervised	PLELog	0.744	0.397	0.518	0.751	0.331	0.461
unsupervised	LogAnomaly	0.565	0.725	0.635	0.523	0.757	0.619
	DeepLog	0.865	0.627	0.727	0.678	0.693	0.685
	LogCluster	0.997	0.700	0.822	0.930	0.700	0.798
	IM	0.158	0.126	0.141	0.140	0.112	0.125
	PCA	0.251	0.163	0.198	0.172	0.140	0.154

SynHDFS, additional experiment results, and source code publicly available under a suitable license upon acceptance.

4.2 RQ1: Fine-tuned GPT-3 Effectiveness

Methodology. To answer RQ1, we evaluate the effectiveness of GPT-3_{FT} compared to ML+DL methods, for ADUL and ADSL. We adopt LOGEVOL-Hadoop as the experiment dataset, since it contains a stable log dataset (Hadoop 2) and an unstable log dataset (Hadoop 3). Concretely, for ADUL, Hadoop 2 is used for training and Hadoop 3 is used for testing, whereas for ADSL Hadoop 2 is used both for training and for testing.

Results. Table 3 shows the comparison results. For the ADSL task, denoted by 2 → 2, GPT-3_{FT} achieves high performance with an average F1-score of 0.998, though slightly lower (0.001) than the highest F1-score (CNN). We observe that semi-supervised and unsupervised baselines perform significantly worse than supervised methods. Such a decrease in effectiveness is expected since semi-supervised and un-

supervised deep learning baselines are trained with less labeled data. As discussed in § 4.1.3, we apply Fisher’s Exact Test to determine the statistical significance of the differences between GPT-3_{FT} and ML+DL methods. The results indicate no significant differences between GPT-3_{FT} and supervised baselines, i.e., NeuralLog, LogRobust, and CNN, while GPT-3_{FT} significantly outperforms all semi-supervised and supervised baselines.

Regarding ADUL, denoted by 2→3, GPT-3_{FT} outperforms the baselines with a slightly higher F1-score of 0.989. Fisher’s Exact Test confirms that these differences are statistically significant, with all *p*-value below 0.05. The F1 score of all ML+DL methods, except LogRobust’s, worsen with an approximate difference ranging from 0.03 (on NeuralLog) to 0.05 (on PLELog). Comparing LogRobust and GPT-3_{FT}, LogRobust achieves similar Recall, while GPT-3_{FT} achieves a higher precision of 0.98 compared to 0.963. Similar recall and lower precision values indicate a higher number of false positives; since false positives cause false alarms that create excessive work for system monitoring, they tend to be more problematic in practice compared to false negatives (25). Semi-supervised (PLELog) and unsupervised baselines (LogAnomaly, DeepLog, LogCluster, IM, and PCA) generally exhibit lower performance than supervised methods, with all F1-scores falling below 0.83. Moreover, their effectiveness deteriorates for ADUL when compared to ADSL (all F1-scores falling below 0.8). Such decline could be explained by the higher complexity and less predictable nature of unstable log environments, which present more significant challenges for models that operate without the guidance of direct supervision (e.g., semi-supervised and unsupervised models).

In conclusion, GPT-3_{FT} significantly outperforms ML+DL methods for ADUL while the differences for ADSL are not statistically significant compared to the best ML+DL methods. We believe that the ef-

fectiveness of GPT-3_{FT} can be partially attributed to the pre-training of large language models on extensive datasets, which facilitates a robust understanding of diverse patterns and contextual information, setting it apart from ML+DL methods.

The answer to RQ1 is that GPT-3_{FT} fares better than ML+DL methods for ADUL, though it is unclear whether the differences between GPT-3_{FT} and supervised baselines are practically significant. The semi-supervised and unsupervised methods perform significantly worse compared to supervised methods for both ADUL and ADSL.

4.3 RQ2: Impact of Log Evolution

Methodology. To answer RQ2, we compare GPT-3_{FT} with ML+DL methods under different percentages of log changes. As described in § 4.1.1, we performed the experiments under five injection ratios. The injection ratio of 0% represents the stable HDFS dataset, when no changes were applied to it. As for unstable logs, we use SynHDFS, which includes four different injection ratios, namely 5%, 10%, 20%, and 30%. Additionally, we study the individual influence of each type of log change, namely “remove”, “duplicate”, and “shuffle”, as presented in Table 5. To that end, we compare the effectiveness of GPT-3_{FT} and ML+DL methods on the subsets of SynHDFS matching each type.

Results. Table 4 shows each model’s Precision, Recall, and F1-score results on SynHDFS under the above settings: column “IR” indicates the injection ratio namely 0%, 5%, 10%, 20%, 30%; column “M” indicates the evaluation metric; columns “S”, “Semi-S”, and “U” stand for supervised, semi-supervised, and unsupervised models.

We observe that under small injection ratios (until 20%), GPT-3_{FT} fares slightly better, with a differ-

ence of 0.01 in F1-score compared to best ML+DL methods. However, when the injection ratio reaches 30%, there is a minimum difference of approximately 0.05 in precision between GPT-3_{FT} and ML+DL methods. We conducted Fisher’s Exact Test and found statistically significant differences in most comparisons, except when comparing GPT-3_{FT} to NeuralLog and LogRobust at a 0% injection ratio, and to CNN at 5% and 20% injection ratios. In other words, as the injection ratio increases, thereby injecting more log changes into the dataset, the effectiveness gap between GPT-3_{FT} and ML+DL methods models tends to widen. Similar to RQ1, we hypothesize that this phenomenon is due to GPT-3_{FT}’s extensive pre-training on diverse datasets, which enhances its robustness to changes in logs. Nevertheless, it raises an intriguing point for further investigation: determining the upper limits of injection ratios that GPT-3_{FT} can handle and whether the effectiveness gap will continue to expand under these conditions.

Table 5 reports the evaluation results for each individual log change type. We adopt the same acronyms as in Table 4. According to this table, GPT-3_{FT} achieves the highest performance when the injection change is of type “remove” or “duplicate”. However, GPT-3_{FT} is not the most effective model (third place) when the injected change is of type “shuffle”, where LogRobust and NeuralLog fare better. Fisher’s Exact Test results confirm the significance of all the differences, except GPT-3_{FT} vs. CNN with the “duplicate” log change type. We posit that GPT-3, primarily designed for generating and understanding languages based on sequences, is less sensitive to the specific order of inputs, which can be a disadvantage in ADUL where order disruptions are key indicators of anomalies. This characteristic might make it less effective at recognizing anomalies specifically induced by shuffling, as it does not heavily penalize alterations in sequence order. In contrast, NeuralLog

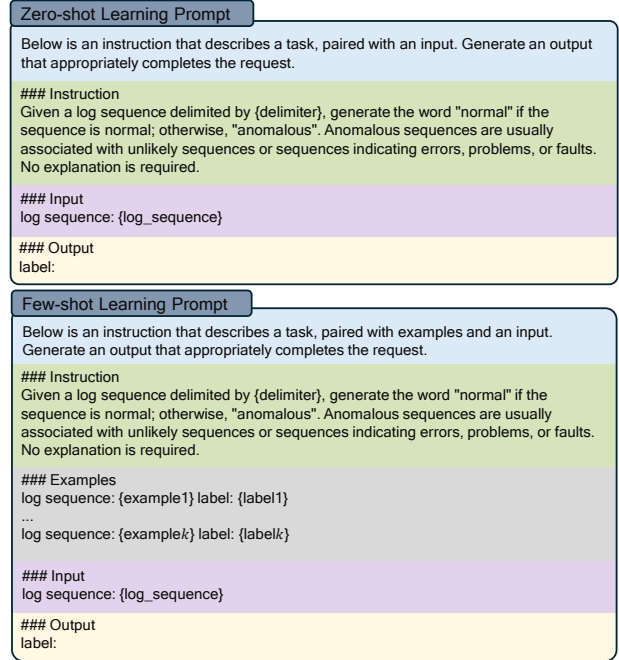


Figure 4: Prompt Designs

and LogRobust are designed specifically for anomaly detection tasks, possibly through enhanced sensitivity to log order and structure.

The answer to RQ2 is that GPT-3_{FT} significantly outperforms all ML+DL methods at injection ratios of 5%, 10%, 20%, 30%, except CNN with ratios of 5% and 20%. GPT-3_{FT} is generally robust to different types of log changes, including “shuffle”, “duplication”, and “removal” of log templates in log sequences. However, changes of type “Shuffle” have the biggest negative impact on the effectiveness of GPT-3_{FT}.

4.4 RQ3: Fine-tuning vs. Prompt engineering

Methodology. To answer RQ3, we explored on two prevalent prompt engineering strategies (62),

Table 4: Effectiveness Comparisons of GPT-3_{FT} and ML+DL methods for Different Injection Ratios

IR	M	S			Semi-S		U			IM	PCA
		GPT-3 _{FT}	NeuralLog	LogRobust	CNN	PLELog	LogAnomaly	DeepLog	LogCluster		
0%	P	0.994	0.981	0.993	0.995	0.952	0.989	0.989	1.000	0.567	0.978
	R	0.993	0.996	0.996	0.996	0.948	0.843	0.827	0.712	0.651	0.629
	F1	0.996	0.988	0.995	0.995	0.950	0.910	0.900	0.832	0.606	0.765
5%	P	0.978	0.948	0.914	0.962	0.534	0.417	0.427	0.931	0.241	0.753
	R	0.995	0.995	0.990	0.998	0.975	0.819	0.848	0.707	0.651	0.643
	F1	0.988	0.971	0.951	0.979	0.689	0.553	0.568	0.803	0.351	0.694
10%	P	0.961	0.956	0.914	0.944	0.312	0.260	0.257	0.885	0.152	0.591
	R	0.997	0.975	0.983	0.993	0.984	0.833	0.774	0.708	0.651	0.643
	F1	0.978	0.965	0.947	0.968	0.448	0.396	0.386	0.786	0.247	0.616
20%	P	0.927	0.637	0.890	0.913	0.201	0.156	0.158	0.783	0.087	0.424
	R	0.989	0.990	0.991	0.979	0.978	0.846	0.794	0.707	0.651	0.646
	F1	0.957	0.775	0.938	0.944	0.333	0.263	0.264	0.743	0.154	0.512
30%	P	0.896	0.550	0.795	0.781	0.141	0.112	0.114	0.701	0.062	0.340
	R	0.985	0.991	0.987	0.970	0.970	0.832	0.831	0.710	0.651	0.648
	F1	0.938	0.707	0.881	0.865	0.240	0.197	0.201	0.705	0.113	0.446

namely zero-shot and few-shot learning, with the same dataset settings used to address RQ1 and RQ2. As mentioned at the beginning of § 4, we adopt GPT-4 which is the most advanced GPT model available for prompt engineering (denoted as GPT-4_{PE}).

For designing prompts, we followed the tactics reported by Winteringham (70). Prompts are structured into four to five parts: *Initial description*, *Instruction*, *Examples* (for few-shot learning only), *Input*, and *Output*. Figure 4 provides examples of zero-shot and few-shot learning prompts. The initial description is used to make GPT familiarize with the subsequent components of the prompt. The instruction part is designed to introduce the task by describing its input format, desired output format, and goal. The *{delimiter}* is a token to be replaced by the sequence delimiter (such as brackets). We empirically studied different instructions and present

the one yielding the best effectiveness in Figure 4. At the end of the instruction part, we add the sentence “No explanation is required” to ensure that the response only includes the desired output label, to help automate the evaluation of results without post-processing. The example part includes a few pairs of input and output sampled from the training set so that GPT can understand the contextual information regarding the task, explained further below. In the input part, the *{log_sequence}* token is to be replaced by the input log sequence. Lastly, the output section guides GPT in predicting the label of the input log sequence. This is done by providing GPT-4 with a formatted prompt that ends with “label:”, where GPT-4 is expected to generate the next token as either “normal” or “anomalous”, depending on its analysis of the log sequence.

The salient feature of few-shot learning prompt

Table 5: Effectiveness Comparisons of GPT-based Models and Baselines for Different Change Types on SynHDFS

Model	No Change			remove			duplicate			shuffle		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
GPT-3 _{FT}	0.994	0.993	0.996	0.983	0.999	0.991	0.9722	0.986	0.979	0.912	0.999	0.953
NerualLog	0.981	0.996	0.988	0.979	0.987	0.981	0.7059	0.994	0.825	0.980	0.991	0.985
LogRobust	0.993	0.996	0.995	0.979	0.985	0.982	0.766	0.976	0.858	0.986	0.995	0.990
CNN	0.995	0.996	0.995	0.983	0.983	0.983	0.959	0.988	0.973	0.789	0.992	0.879
PLELog	0.952	0.948	0.950	0.644	0.965	0.772	0.217	0.983	0.354	0.530	0.985	0.683
LogAnomaly	0.989	0.843	0.910	0.465	0.823	0.594	0.157	0.911	0.268	0.378	0.822	0.518
DeepLog	0.989	0.827	0.900	0.438	0.815	0.569	0.159	0.904	0.270	0.365	0.810	0.503
LogCluster	1.000	0.712	0.832	1.000	0.707	0.828	0.701	0.707	0.705	1.000	0.707	0.828
IM	0.567	0.651	0.606	0.106	0.651	0.183	0.117	0.651	0.199	0.567	0.651	0.606
PCA	0.978	0.629	0.765	0.975	0.639	0.772	0.340	0.646	0.446	0.974	0.641	0.773

compared to zero-shot learning is its incorporation of examples in the prompt. As exemplified in Figure 4, the few-shot learning prompt includes k examples and their labels, denoted by $\{example_i\}$ and $\{label_i\}$, respectively. We set k to 10 given the models’ maximum input token limit, which we discuss further in § 4.5. For each dataset, instead of random selection, we selected samples from its training set by applying the few-shot sampling algorithm based on Mean Shift Clustering (4). This algorithm was introduced in a recent study by Ma et al. (49), to increase the diversity of selected samples. We evaluated these prompts on LOGEVOL-Hadoop and SynHDFS for ADUL.

Results. Table 6 shows Precision, Recall, and F1-score results of GPT-3_{FT} and GPT-4_{PE}. “V/IR” indicates two main experiment settings: training and testing versions (“V”) for Hadoop or injection ratio (“IR”) for SynHDFS.

As presented in Table 6, GPT-3_{FT} outperforms GPT-4_{PE} for all datasets, with minimum differences in F1-score of 0.062 and 0.33 for LOGEVOL-Hadoop and SynHDFS, respectively. Fisher’s Exact

Test results confirm the statistical significance of all the differences ($p_{value} < 0.05$). One plausible reason for such performance difference is prompt engineering’s inclination to overfit on the examples/context provided in the prompt. Optimal effectiveness with prompt engineering relies on the examples in the prompt aligning with the true distribution of the log data, which is commonly observed in stable logs. However, in unstable logs, there is a risk of sampling non-representative examples due to the evolving nature of the logs, hence leading to degraded effectiveness in anomaly detection.

We also remark that few-shot prompting yields significantly better effectiveness than zero-shot prompting ($p_{value} < 0.05$ using Fisher’s Exact Test) with differences in F1-score higher than 0.4. Few-shot prompting yields F1-scores above 0.9 for LOGEVOL-Hadoop and above 0.51 for SynHDFS. The F1-score of zero-shot prompting remains under 0.46 and 0.08 on LOGEVOL-Hadoop and synHDFS, respectively. We believe the superiority demonstrated by few-shot prompting is due to its incorporation of examples. In contrast, in zero-shot prompting, pre-trained models are directly evaluated on un-

Table 6: Effectiveness Comparisons of Fine-tuned (GPT-3) and Prompt Engineered (GPT-4) Models on Different Unstable Log

Dataset	V/IR	M	FT	PE _{zero-shot}	PE _{few-shot}
			GPT-3	GPT-4	GPT-4
LOGEVOL-Hadoop	2 → 2	P	0.997	0.322	0.889
		R	0.999	0.416	0.988
		F1	0.998	0.454	0.936
	2 → 3	P	0.980	0.270	0.834
		R	0.999	0.771	0.988
		F1	0.989	0.400	0.905
SynHDFS	0%	P	0.994	0.040	0.686
		R	0.993	0.972	0.647
		F1	0.996	0.078	0.666
	5%	P	0.978	0.027	0.550
		R	0.995	0.972	0.662
		F1	0.988	0.052	0.600
	10%	P	0.961	0.026	0.512
		R	0.997	0.972	0.663
		F1	0.978	0.052	0.578
	20%	P	0.927	0.026	0.457
		R	0.989	0.966	0.669
		F1	0.957	0.050	0.543
	30%	P	0.896	0.025	0.415
		R	0.985	0.964	0.675
		F1	0.938	0.050	0.514
	remove	P	0.983	0.026	0.562
		R	0.999	0.970	0.664
		F1	0.991	0.052	0.609
	duplicate	P	0.972	0.026	0.448
		R	0.986	0.962	0.671
		F1	0.979	0.051	0.537
	shuffle	P	0.912	0.026	0.569
		R	0.999	0.972	0.666
		F1	0.953	0.052	0.613

stable logs without exposure to any examples from the stable logs or unstable logs. Another intriguing observation is that, for highly unstable SynHDFS logs (injection ratio of 30 %), the F1-scores of few-shot prompting, although lower than those of supervised baselines, are still higher than those of semi-supervised and most unsupervised baselines (such as PLELog, logAnomaly, and DeepLog), as reported in RQ1 and RQ2. This result highlights the potential usefulness of few-shot prompting for ADUL when

limited labeled data is available.

The answer to RQ3 is that GPT-3_{FT} is significantly more effective than GPT-4_{PE} on unstable logs. Moreover, GPT-4_{PE} using few-shot prompting is significantly more effective than GPT-4_{PE} using zero-shot prompting and semi-supervised and unsupervised ML+DL methods. This suggests it is advantageous to use GPT-4_{PE} when limited labeled data is available for fine-tuning.

4.5 Discussion: Challenges and Opportunities

In this section, to guide software engineers, we highlight the challenges and opportunities for using LLMs for ADUL.

Adaptability. One major challenge lies in the length variability of log sequences, potentially exceeding the input token limits of LLMs. Compared to prompt engineering, fine-tuning offers an advantage in tackling this challenge. Specifically, the prompt for fine-tuning tends to be shorter than that for prompt engineering. The fine-tuning prompt comprises only the log sequences. This contrasts with prompts for prompt engineering, especially few-shot prompting, which requires additional tokens for task instruction and example descriptions, thereby exacerbating the token limit issue. However, in the future, the adaptability of GPT-3_{FT} can be challenged by longer sequences or models with smaller token limits. Therefore, further investigation is needed to develop strategies for enhancing LLM’s capability in handling long sequences, such as prompt compression (41) and LLM context extension (3).

Resource Consumption and Financial Cost. The second challenge faced by GPT-3_{FT} is its high demand for computing resources, especially in the *fine-tuning phase*. Customizing the pre-trained GPT-3’s weights typically requires substantial computational

resources, taking minutes to hours, depending on the model and dataset size. We acknowledge that such a resource consumption is not negligible, which incurs considerable financial costs. The total cost of using OpenAI APIs in this study, including preliminary experiments, fine-tuning with GPT-3 and GPT-3.5, and prompt engineering with GPT-3.5 and GPT-4, adds up to around 3,000 USD. Therefore, the feasibility and value of exploring GPT models in real-world applications requires further evaluation, tailored to the specific conditions and constraints of practitioners.

Fine-tuning Constraints. Despite the state-of-the-art performance demonstrated by GPT-3_{FT}, its fine-tuning process is limited to the OpenAI APIs. Specifically, popular GPT models, such as GPT-3, GPT-3.5, and GPT-4, are closed-sourced models, disallowing any modification of their underlying architecture. However, numerous open-source LLMs (e.g., LLama (65)) and advanced fine-tuning techniques may improve the performance of ADUL. In particular, PEFT techniques (50) are gaining increasing attention, due to their efficiency in adapting LLMs to various downstream tasks. The key idea of PEFT is to construct a low-dimensional reparameterization of LLMs for fine-tuning, while achieving comparable performance to full fine-tuning. We are confident that applying various PEFT techniques in the future, such as CoDA (39), U-Diff (13) and LoRa (23) on open-source LLMs with similar performance to GPT-3, can further improve ADUL performance.

4.6 Threats to Validity

Internal validity. The lack of transparency regarding the pre-training data of GPT-3 and GPT-4 raises concerns about potential data leakage, wherein our testing data might have been inadvertently included in the pre-training corpus. This could result in unrealistically positive results. Notably, the HDFS

dataset, a key evaluation dataset in this study, could potentially be part of the pre-training corpus. To mitigate this possibility and ensure the validity of our experiment results, we incorporated two additional datasets: LOGEVOL and SynHDFS. LOGEVOL was curated after the training and release of GPT-3 and GPT-4, while SynHDFS was synthesized specifically for this study.

Conclusion Validity. It is widely acknowledged that LLMs often produce non-deterministic responses even when provided with identical prompts, posing a potential threat to the conclusion validity of GPT-3_{FT}. To address this challenge, as detailed in § 4.1.4, we mitigate the risk by configuring the LLMs to generate responses with minimal randomness, achieved by setting the temperature parameter to 0. To further reduce the influence of randomness from our evaluation results, as discussed in § 4.1.4, we run each experiment five times and calculated the mean value as the final result. While alternative strategies for alleviating randomness exist, such as employing ensemble LLMs, their implementation typically demands substantial computational resources, significantly inflating experiment costs. However, future work should include exploring these strategies, particularly with other low-cost, open-source LLMs.

External Validity. GPT-3_{FT} aims to perform accurate anomaly detection for unstable logs resulting from software evolution. However, it is anticipated that significant performance degradation may occur when the gap between two software versions is extensive. For instance, the introduction of YARN for job management in Hadoop 2 brings about substantial alterations to the Hadoop architecture and consequently results in significant changes in the logs (68). In such cases, GPT-3_{FT} may not be applicable. To address this potential limitation, we studied the impact of software evolution gaps by varying the injection rate. Experimental results demonstrate that GPT-3_{FT} maintains acceptable perfor-

mance (F1-score of 0.9384) even at an injection rate of 30%, which means 30% of the log sequences have been altered. However, it is uncommon for software updates to cause such extensive changes in logs, except for major versions (61), and we thus expect the conclusions from this study to be valid in most evolution scenarios.

5 Related Work

5.1 Anomaly Detection on Unstable Logs

Log-based anomaly detection has been extensively studied in the literature to enhance the dependability of software-intensive systems (36). However, only a few studies have investigated anomaly detection on unstable logs derived from software evolution (77; 40; 24), a common situation in practice. Zhang et al. (77) first identified such a challenge and proposed LogRobust (see § 4.1.2) to leverage an attention-based Bi-LSTM as an anomaly detector. They also created a new unstable log dataset called Synthetic HDFS to evaluate the effectiveness and robustness of LogRobust. This inspired a number of follow-up works, including HitAnomaly (24) and SwissLog (40). SwissLog adopts the same architecture (i.e., Bi-LSTM) as LogRobust and aims to further improve it by incorporating time embeddings and Bert-based semantic embeddings. HitAnomaly, however, leverages a much larger model based on a hierarchical transformer architecture. The high complexity of the HitAnomaly model allows it to tackle not only the static parts of log messages, but also dynamic parts, such as numerical values that have been masked in the log templates. Experiment results demonstrate the superiority of HitAnomaly on stable logs compared to LogRobust, while showing a robust performance for only a small injection ratio (under 20%) of unstable logs and being outperformed by LogRobust from 20% to 30%.

Huo et al. (25) proposed EvLog, explicitly targeting the identification of fault-indicating log messages within a log sequence in an unsupervised manner. EvLog leverages a multi-level semantics extractor and attention mechanism to handle unstable logs and demonstrated better results than its unsupervised baselines. As part of the EvLog study, Huo et al. (25) also introduced two real-world unstable logs datasets: LOGEVOL-Hadoop and LOGEVOL-Spark. Following their work, we have comprehensively evaluated LogTune’s performance for unstable log anomaly detection on both synthetic datasets (SynHDFS) and real datasets (LOGEVOL-Hadoop) (see § 4.1.1).

5.2 Application of LLMs on Log Analysis

Over the past few years, LLMs have been widely adopted on different log-related software engineering tasks to enhance effectiveness and generalizability, including log-based anomaly detection and log parsing (44; 49; 71).

In *anomaly detection*, the application of LLMs started by leveraging BERT (11) to capture contextual information of logs with semantic-based representations. LogBERT (14) utilizes BERT to learn the semantics of normal log messages using self-supervised tasks and predict an anomaly where log messages of an input sequence deviate from the normal distribution. Le and Zhang (34) proposed Neurallog (see § 4.1.2), which leverages BERT as a semantic-based log message representation and outperforms its similar scale LLMs for log message representation, including RoBERTa (9) and GPT-2 (59). Han et al. (15) introduced LogGPT, which leverages the reinforcement learning strategy to fine-tune GPT-2 for log-based anomaly detection. More recently, Liu et al. (44) proposed LogPrompt, which adopts LLMs such as GPT-3 and Vicuna (5) for online log parsing and anomaly detection using prompt engi-

neering strategies. We have not included LogPrompt in our evaluation since it was published while the work reported in this paper was being finalized. Nevertheless, as part of RQ3, we have compared—in terms of zero-shot and few-shot learning—prompt engineering approaches that are similar to Log-Prompt.

In *log parsing*, Le and Zhang (35) first explored the application of ChatGPT, a chatbot based on LLMs, on log parsing using prompt engineering, and achieved promising results with zero-shot and few-shot prompts. Le and Zhang (37) introduced LogPPT, which adopts prompt-based few-shot learning using RoBERTa for log parsing, outperforming the state-of-the-art log parsers on 16 datasets. Xu et al. (71) proposed DivLog, a few-shot learning-based method that selects five appropriate labeled candidates as examples for each target log and constructs them into a prompt. Evaluation results show that DivLog achieves state-of-the-art performance on six public datasets. Ma et al. (49) investigated the application of prompt engineering using open-source LLMs such as Flan-T5 (8) and LLaMA (65) on log parsing. Similar to our findings, they demonstrate that overall fine-tuning, even with few samples, achieves better results than in-context learning (similar to few-shot learning in RQ3). Very recently, Jiang et al. (28) proposed LILAC, which leverages LLMs for log parsing with an adaptive parsing cache to reduce the query times to LLMs by several orders.

6 Conclusion and future work

This paper investigated the performance of GPT-3_{FT} (fine-tuned GPT-3) for Anomaly Detection on Unstable Logs (ADUL) in the context of software evolution. Our extensive experiments show that GPT-3_{FT} is effective, outperforming existing baselines (including supervised, unsupervised, and semi-

supervised models) with an F1-score of 0.9895. However, differences with supervised alternatives may not always be practically significant and worth the additional cost entailed by large models like GPT-3. Additionally, GPT-3_{FT} remains effective (F1-score > 0.93) when up to 30 % of the log sequences are affected by changes. Last, when exploring prompt engineering with GPT-4, as an alternative to fine-tuning, GPT-3_{FT} appears to be significantly better, thus suggesting that fine-tuning is a preferable option when adequate labelled data is available.

In the future, we plan to explore other promising LLMs for anomaly detection on unstable logs, including GPT-4 (currently publicly inaccessible for fine-tuning) and open-source LLMs such as Gemma (64) and Llama 2 (65). It is worth noting that applying public LLMs requires advanced fine-tuning methods, such as Parameter-efficient Fine-Tuning (PEFT) (50) to reduce the cost of computational resources, which can be explored in future research.

Acknowledgments

This work was supported by the Canada Research Chair and Discovery Grant programs of the Natural Sciences and Engineering Research Council of Canada (NSERC), by a University of Luxembourg’s joint research program grant, the Science Foundation Ireland under Grant 13/RC/2094-2, and by the Luxembourg National Research Fund (FNR), grant reference C22/IS/17373407/LOGODOR. For the purpose of open access, and in fulfillment of the obligations arising from the grant agreement, the authors have applied a Creative Commons Attribution 4.0 International (CC BY 4.0) license to any Author Accepted Manuscript version arising from this submission. The experiments conducted in this work were enabled in part by Digital Alliance of Canada (al-

liancecan.ca).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- [4] Yizong Cheng. Mean shift, mode seeking, and clustering. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 17(8):790–799, 1995. doi: 10.1109/34.400568.
- [5] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- [6] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, pages 1724–1734. ACL, 2014. doi: 10.3115/V1/D14-1179. URL <https://doi.org/10.3115/v1/d14-1179>.
- [7] Guohao Chu, Jiaqi Wang, Qi Qi, Haiyang Sun, Shengtao Tao, and Jun Liao. Prefix-graph: A versatile log parsing approach merging prefix tree with probabilistic graph. In *Proceedings of the 37th International Conference on Data Engineering (ICDE)*, pages 2411–2422, Virtual Event, 2021. IEEE. URL <https://ieeexplore.ieee.org/document/9458609>.
- [8] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models, 2022.
- [9] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoy-

- anov. Unsupervised cross-lingual representation learning at scale, 2020.
- [10] Scott Deerwester, Susan T Dumais, George W Furnas, Thomas K Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American society for information science*, 41(6):391–407, 1990.
 - [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://doi.org/10.18653/v1/n19-1423>.
 - [12] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning, 2017. ISSN 15437221.
 - [13] Demi Guo, Alexander Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4884–4896, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.378. URL <https://aclanthology.org/2021.acl-long.378>.
 - [14] Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2021. doi: 10.1109/IJCNN52387.2021.9534113.
 - [15] Xiao Han, Shuhan Yuan, and Mohamed Trabelsi. Loggpt: Log anomaly detection via gpt. In *2023 IEEE International Conference on Big Data (BigData)*, pages 1117–1122, 2023. doi: 10.1109/BigData59044.2023.10386543.
 - [16] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey, 2024.
 - [17] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. Drain: An online log parsing approach with fixed depth tree, 2017.
 - [18] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. Experience report: System log analysis for anomaly detection, 2016. URL <https://doi.org/10.1109/ISSRE.2016.21>.
 - [19] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. Loghub: A Large Collection of System Log Datasets Towards Automated Log Analytics. *CoRR*, abs/2008.06448, 2020. URL <https://arxiv.org/abs/2008.06448>.
 - [20] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey on automated log analysis for reliability engineering. *ACM Comput. Surv.*, 54(6), jul 2021. ISSN 0360-0300. doi: 10.1145/3460345. URL <https://doi.org/10.1145/3460345>.
 - [21] Maryamsadat Hejazi and Yashwant Prasad Singh. One-class support vector machines approach to anomaly detection. *Applied Artificial Intelligence*, 27(5):351–366, 2013.
 - [22] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, nov 1997. ISSN

- 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [24] Shaohan Huang, Yi Liu, Carol Fung, Rong He, Yining Zhao, Hailong Yang, and Zhongzhi Luan. Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE Transactions on Network and Service Management*, 17(4):2064–2076, 2020. doi: 10.1109/TNSM.2020.3034647.
- [25] Y. Huo, C. Lee, Y. Su, S. Shan, J. Liu, and M. R. Lyu. Evlog: Identifying anomalous logs over software evolution. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 391–402, Los Alamitos, CA, USA, oct 2023. IEEE Computer Society. doi: 10.1109/ISSRE59848.2023.00018. URL <https://doi.ieeecomputersociety.org/10.1109/ISSRE59848.2023.00018>.
- [26] Yintong Huo, Cheryl Lee, Yuxin Su, Shiwen Shan, Jinyang Liu, and Michael R. Lyu. Evlog replication package, 2023. URL <https://github.com/YintongHuo/EvLog>, accessed 2024-05-31.
- [27] Intel. Hibench. <https://github.com/Intel-bigdata/HiBench>, 2021.
- [28] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. Lilac: Log parsing using llms with adaptive parsing cache, 2024.
- [29] Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, H erve J egou, and Tomas Mikolov. Fasttext.zip: Compressing text classification models, 2016.
- [30] Suhas Kabinna, Weiyi Shang, Cor-Paul Bezeemer, and Ahmed E. Hassan. Examining the stability of logging statements. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, volume 1, pages 326–337, 2016. doi: 10.1109/SANER.2016.29.
- [31] Yoon Kim. Convolutional neural networks for sentence classification. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–1751, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1181. URL <https://aclanthology.org/D14-1181>.
- [32] Herb Krasner. The cost of poor software quality in the us: A 2020 report. *Proc. Consortium Inf. Softw. QualityTM (CISQTM)*, pages 1–46, 2021.
- [33] Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, 12:100470, 2023. ISSN 2666-8270. doi: <https://doi.org/10.1016/j.mlwa.2023.100470>. URL <https://www.sciencedirect.com/science/article/pii/S2666827023000233>.
- [34] V. Le and H. Zhang. Log-based anomaly detection without log parsing, nov 2021. URL <https://doi.org/10.1109/ISSRE59848.2023.00018>.

- ieeecomputersociety.org/10.1109/ASE51524.2021.9678773.
- [35] V. Le and H. Zhang. Log parsing: How far can chatgpt go? In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1699–1704, Los Alamitos, CA, USA, sep 2023. IEEE Computer Society. doi: 10.1109/ASE56229.2023.00206. URL <https://doi.ieeecomputersociety.org/10.1109/ASE56229.2023.00206>.
- [36] Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection with deep learning: how far are we? In *Proceedings of the 44th International Conference on Software Engineering, ICSE '22*, page 1356–1367, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510155. URL <https://doi.org/10.1145/3510003.3510155>.
- [37] Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering, ICSE '23*, page 2438–2449. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00204. URL <https://doi.org/10.1109/ICSE48619.2023.00204>.
- [38] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436, 2015.
- [39] Tao Lei, Junwen Bai, Siddhartha Brahma, Joshua Ainslie, Kenton Lee, Yanqi Zhou, Nan Du, Vincent Y. Zhao, Yuexin Wu, Bo Li, Yu Zhang, and Ming-Wei Chang. Conditional adapters: Parameter-efficient transfer learning with fast inference, 2023.
- [40] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults. *Proceedings - International Symposium on Software Reliability Engineering, ISSRE*, 2020-Octob:92–103, 2020. ISSN 10719458. doi: 10.1109/ISSRE5003.2020.00018.
- [41] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.391. URL <https://aclanthology.org/2023.emnlp-main.391>.
- [42] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. Log Clustering Based Problem Identification for Online Service Systems. In *Proceedings of the 38th International Conference on Software Engineering, ICSE - Companion Volume*, pages 102–111, Austin, TX, USA, 2016. ACM. doi: 10.1145/2889160.2889232. URL <https://doi.org/10.1145/2889160.2889232>.
- [43] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008. doi: 10.1109/ICDM.2008.17.
- [44] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xiaofeng Zhao, and Hao Yang. Log-prompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Proceedings*

- of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE-Companion '24, page 364–365, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705021. doi: 10.1145/3639478.3643108. URL <https://doi.org/10.1145/3639478.3643108>.
- [45] Steven Locke, Heng Li, Tse-Hsun Peter Chen, Weiyi Shang, and Wei Liu. Logassist: Assisting log analysis through log summarization. *IEEE Transactions on Software Engineering*, 48(9):3227–3241, 2022. doi: 10.1109/TSE.2021.3083715.
- [46] Jian Guang Lou, Qiang Fu, Shengqi Yang, Ye Xu, and Jiang Li. Mining invariants from console logs for system problem detection, 2019.
- [47] Long Ouyang Jeff Wu Xu Jiang Diogo Almeida Carroll L. Wainwright Pamela Mishkin Paul Christiano Jan Leike Ryan Lowe et al. Gpt-4 technical report. 2023. URL <https://api.semanticscholar.org/CorpusID:257532815>.
- [48] Siyang Lu, Xiang Wei, Yandong Li, and Liqiang Wang. Detecting anomaly in big data system logs using convolutional neural network. *IEEE Access*, 6:21929–21940, 2018. doi: 10.1109/ACCESS.2018.2811530.
- [49] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. Llm-parser: An exploratory study on using large language models for log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3639150. URL <https://doi.org/10.1145/3597503.3639150>.
- [50] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. Peft: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- [51] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.*, 2(11):205, 2017.
- [52] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *IJCAI*, volume 19, pages 4739–4745, 2019.
- [53] Haibo Mi, Huaimin Wang, Yangfan Zhou, Michael Rung-Tsong Lyu, and Hua Cai. Toward fine-grained, unsupervised, scalable performance diagnosis for production cloud computing systems. *IEEE Transactions on Parallel and Distributed Systems*, 24(6):1245–1255, 2013. doi: 10.1109/TPDS.2013.21.
- [54] Digital Research Alliance of Canada. <https://alliancecan.ca/>, 2016. Accessed: March 2, 2023.
- [55] Adam Oliner and Jon Stearley. What supercomputers say: A study of five system logs, 2007.
- [56] OpenAI. Fine-tuning: preparing your dataset, 2024. URL <https://platform.openai.com/docs/>

- [guides/fine-tuning](#), accessed 2024-01-02.
- [57] OpenAI. Openai models, 2024. URL <https://platform.openai.com/docs/models>, accessed 2024-05-10.
- [58] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. Training language models to follow instructions with human feedback. *ArXiv*, abs/2203.02155, 2022. URL <https://api.semanticscholar.org/CorpusID:246426909>.
- [59] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019. URL <https://api.semanticscholar.org/CorpusID:160025533>.
- [60] M. Schuster and K.K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997. doi: 10.1109/78.650093.
- [61] Weiyi Shang, Zhen Ming Jiang, Bram Adams, Ahmed E. Hassan, Michael W. Godfrey, Mohamed Nasser, and Parminder Flora. An exploratory study of the evolution of communicated information about the execution of large software systems. In *2011 18th Working Conference on Reverse Engineering*, pages 335–344, 2011. doi: 10.1109/WCRE.2011.48.
- [62] Jiho Shin. Prompt Engineering or Fine Tuning : An Empirical Software Engineering Tasks, 2024.
- [63] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system, 2010.
- [64] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, and Shreya Pathak. Gemma: Open models based on gemini research and technology, 2024.
- [65] Hugo Touvron, Louis Martin, Kevin Stone, and Peter Albert. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [66] Graham JG Upton. Fisher’s exact test. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 155(3):395–402, 1992.
- [67] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.
- [68] Vinod Kumar Vavilapalli, Arun C Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing*, pages 1–16, 2013.
- [69] Xiaolei Wang, Lin Yang, Dongyang Li, Linru Ma, Yongzhong He, Junchao Xiao, Jiyuan

- Liu, and Yuexiang Yang. Maddc: Multi-scale anomaly detection, diagnosis and correction for discrete event logs. In *Proceedings of the 38th Annual Computer Security Applications Conference, ACSAC '22*, page 769–784, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450397599. doi: 10.1145/3564625.3567972. URL <https://doi.org/10.1145/3564625.3567972>.
- [70] Mark Winteringham. Ai-assisted testing, 2024. Estimated publication in Fall 2024, MEAP began December 2023.
- [71] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. Divlog: Log parsing with prompt enhanced in-context learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3639155. URL <https://doi.org/10.1145/3597503.3639155>.
- [72] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael Jordan. Online system problem detection by mining patterns of console logs, 12 2009.
- [73] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, SOSP '09*, page 117–132, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605587523. doi: 10.1145/1629575.1629587. URL <https://doi.org/10.1145/1629575.1629587>.
- [74] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan. Detecting large-scale system problems by mining console logs, 2010.
- [75] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. Semi-supervised log-based anomaly detection via probabilistic label estimation, 2021.
- [76] T. Zhang, H. Qiu, G. Castellano, M. Rifai, C. Chen, and F. Pianese. System log parsing: A survey. *IEEE Transactions on Knowledge & Data Engineering*, 35(08):8596–8614, aug 2023. ISSN 1558-2191. doi: 10.1109/TKDE.2022.3222417.
- [77] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, Junjie Chen, Xiaoting He, Randolph Yao, Jian Guang Lou, Murali Chintalapati, Furao Shen, and Dongmei Zhang. Robust log-based anomaly detection on unstable log data, 2019.
- [78] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.